EXPERIMENT ASSESSMENT

ACADEMIC YEAR 2025-26

Course: Machine Learning Lab

Course code: CSL604

Year: TE3 SEM: VI

Experiment No. 06
AIM:- To implement McCulloch Pitt Model.
Name: Satyam Maurya
Roll Number: 62
Date of Performance: 23/2
Date of Submission: 9/3

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission.	10	
Total	**20**	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	5	3	2
Understanding	5	3	2
Journal work and timely submission.	10	8	4

Checked by

Name of Faculty : Ms. Nivedha Raut

Signature :

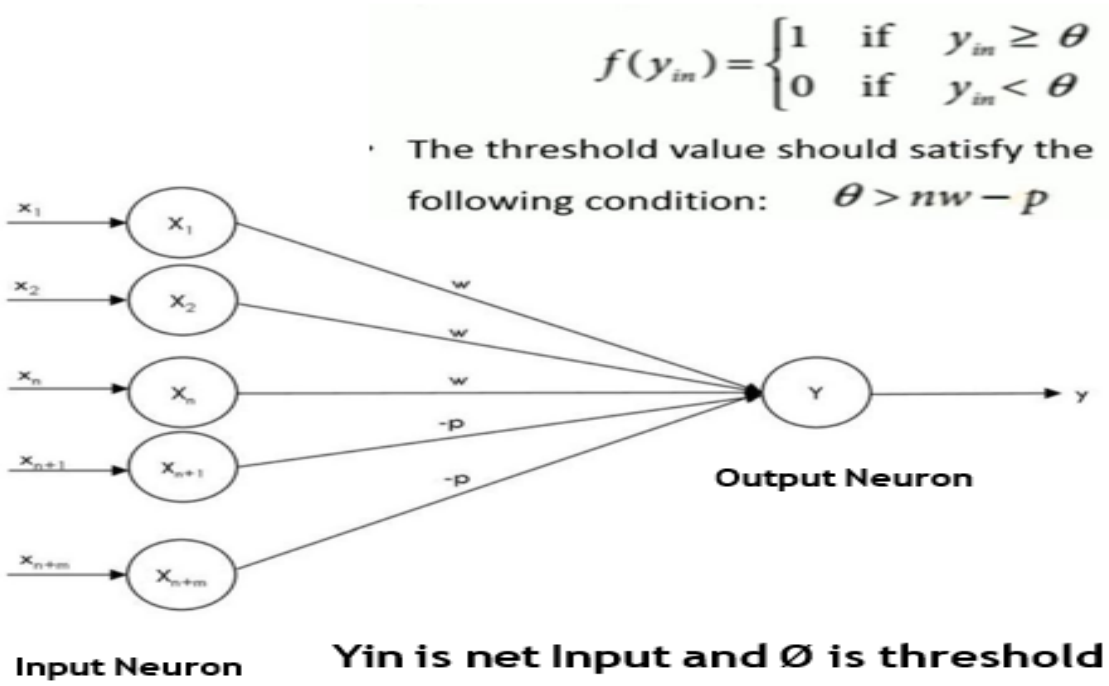
Date :

Aim - To implement McCulloch Pitt Model.

Objective - To simulate the biological process of learning by implementing a McCulloch Pitt Model.

Theory –

The McCulloch-Pitts neuron was the earliest neural network discovered in 1943. It is usually called as M-P neuron. The M-P neurons are **connected by directed weighted paths**. It should be noted that the **activation of M-P neuron is binary**, that is,at any time step the neuron may fire or may not fire. The weights associated with the communication links may be **excitatory (weight is positive) or inhibitory (weight is negative). The threshold plays a major role in M-P neuron**: There is a fixed threshold for each neuron, and if the net input to the neuro is greater than then threshold then the neuron fires.



- **Binary Output:** The model produces a binary output, which means it can be either active (firing) with a value of 1 or inactive (not firing) with a value of 0.
- **Inputs and Weights:** It takes multiple binary inputs, each with its associated weight. The inputs can be either 0 or 1, and the weights represent the strength of the connection between inputs and the neuron.

- **Threshold (Ø) :** The model includes a threshold value (often denoted as "θ". The neuron will only fire (output 1) if the weighted sum of its inputs exceeds this threshold.

Code:

```
w1 = int(input("Enter weight w1: "))
w2 = int(input("Enter weight w2: "))
threshold = int(input("Enter threshold value: "))

# Number of input patterns
n = int(input("Enter number of input patterns: "))

for i in range(n):
    print(f"\nPattern {i+1}")
    x1 = int(input("Enter x1 (0 or 1): "))
    x2 = int(input("Enter x2 (0 or 1): "))

    # Calculate net input
    net = x1*w1 + x2*w2

    if net >= threshold:
        output = 1
    else:
        output = 0

    print("Output:", output)
```

Output:

```
Enter weight w1: 1
Enter weight w2: 1
Enter threshold value: 2
Enter number of input patterns: 4

Pattern 1
Enter x1 (0 or 1): 0
Enter x2 (0 or 1): 0
Output: 0

Pattern 2
Enter x1 (0 or 1): 0
```

Enter x2 (0 or 1): 1
Output: 0

Pattern 3
Enter x1 (0 or 1): 1
Enter x2 (0 or 1): 0
Output: 0

Pattern 4
Enter x1 (0 or 1): 1
Enter x2 (0 or 1): 1
Output: 1

Conclusion:

Q1. What is the McCulloch-Pitts model and how does it work?

The McCulloch-Pitts model is a simple mathematical model of an artificial neuron that operates on a binary threshold mechanism. It works by taking multiple binary inputs, multiplying them with corresponding weights, and calculating their weighted sum. This sum is then compared with a predefined threshold value. If the weighted sum is greater than or equal to the threshold, the neuron produces an output of 1; otherwise, it produces 0. This behavior mimics the firing mechanism of biological neurons and is commonly used to implement basic logical operations.

Q2. What are the limitations of the McCulloch-Pitts model?

The McCulloch-Pitts model has several limitations, as it can only handle binary inputs and outputs and is restricted to solving linearly separable problems. It does not have the capability to learn from data because weights and threshold values are fixed and not updated dynamically. Additionally, it cannot solve complex problems such as the XOR function and lacks flexibility, making it less suitable for modern machine learning applications